Bahash-AI at ArGuard Shared Tasks: Multimodal Ensemble Learning for Hateful Meme Detection

Ahmadullah Laskar
Department of AI and DS
KL University
Andhra Pradesh, India
Bahash-AI, Bahash Private Limited
Silchar, India
ahmadullahlaskar2004@gmail.com

Sahinur Rahman Laskar
School of Computer Science
UPES
Dehradun, India
sahinurlaskar.nits@gmail.com

Abstract

Hateful content in memes is difficult to detect automatically, as harmful intent often emerges from the interaction between image and text rather than either modality alone, a challenge compounded in Arabic by dialectal variation and sarcasm. This paper presents the system submitted by team Bahash-AI to Subtask A1 (binary hateful meme detection) of the ArGuard shared task at ArabicNLP 2026. A late-fusion classifier combining MARBERTv2 and ViT-base is first developed and evaluated as a standalone development-phase system. Building on this, the final submission ensembles a CLIP-based text-image classifier with this same late-fusion classifier, retrained on the combined train, development, and development-test partitions. Class imbalance (approximately 30% Hateful) is addressed via class-weighted losses, oversampling, and threshold selection. The final ensemble achieves a Macro-F1 of 0.7457 and an accuracy of 0.7680 on the blind test set, ranking 12th of 16 teams on the official leaderboard and outperforming the development-phase system by 9 Macro-F1 points. Error analysis using per-class precision and recall shows that the two systems reach a similar Hateful-class F1 gap through opposite error patterns.

1 Introduction

Hateful content in memes is difficult to detect automatically, as harmful meaning frequently arises from the interaction between an image and its accompanying text rather than from either modality in isolation, a challenge first demonstrated at scale by the Hateful Memes Challenge (Kiela et al., 2020). This challenge is compounded for Arabic content, where dialectal variation, code-switching, sarcasm, and culturally specific references make the task substantially harder than in high-resource languages. The ArGuard shared task at ArabicNLP 2026 (Alam et al., 2026) addresses this problem through Subtask A1, a binary classification task over Arabic memes drawn from the AHA-Memes benchmark (Kmainasi et al., 2026).

We participated in Subtask A1 as team Bahash-AI, building two related multimodal systems and using development-phase results to inform our final submission. The main contributions of this paper are as follows.

- A late-fusion architecture combining MARBERTv2 text embeddings with ViT-base (Dosovitskiy et al., 2021) image embeddings via concatenation, trained with class-weighted cross-entropy and minority-class oversampling, establishing a development-phase baseline of 0.6554 Macro-F1 on the development-test split.
- A two-branch ensemble for our final submission, retaining this late-fusion classifier as one branch and adding a second, CLIP-based (Radford et al., 2021) branch that fuses text and image embeddings through explicit interaction terms, with both branches retrained on the union of all released labelled partitions.
- Class-imbalance handling through class-weighted losses, minority-class oversampling, and threshold selection, discussed with the caution that no controlled ablation isolates each technique's individual contribution.
- An error analysis, grounded in per-class precision and recall from the official scorer, characterizing the gap between development- and final-phase results and identifying a precision-recall trade-off on the Hateful class rather than a single shared bottleneck.

2 Data and Preprocessing

We use the official ArGuard Subtask A1 data splits, distributed as JSON Lines files with `id`, `text`,

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 432–436
October 28-29, 2026 ©2026 Association for Computational Linguistics

Split	Instances
Train	3,500
Development	500
Development-test	500
Test (blind)	500

Table 1: ArGuard Subtask A1 data splits.

`image_path`, and `label` fields, summarized in Table 1. The `train` split contains 3,500 labelled memes, `dev` and `dev_test` each contain 500 labelled memes, and the blind `test` split contains 500 unlabelled memes on which official leaderboard scores are computed. Labels are binary, *Hateful* or *Not Hateful*, with Not Hateful making up roughly 70% of the labelled data; we treat this as a class-imbalanced classification problem throughout (He and Garcia, 2009).

The `text` field is the caption or overlay text provided by the organizers; no additional OCR is performed. Text is tokenized with the matching tokenizer for each encoder, truncated or padded to 128 tokens for MARBERT and 77 tokens for CLIP. Images are converted to RGB and passed through each encoder's native processor; on load failure, a blank placeholder image is substituted so that batches remain well formed. No external datasets are used beyond the pretrained encoders described in Section 3.

3 System Architecture and Training

Two systems are described in the order they were developed. The development-phase system is a standalone late-fusion classifier evaluated on the `dev_test` split; this same architecture forms Branch 2 of the final ensemble, to which a CLIP-based branch is added and which is retrained on additional data for the final blind test submission. The two systems are therefore related rather than independent. Figure 1 illustrates the architecture of the final submitted system.

3.1 Development-Phase System: Late-Fusion Classifier

The development-phase system encodes text and image independently and combines them by concatenation. Text is encoded with MARBERTv2 (Abdul-Mageed et al., 2021), a BERT-based (Devlin et al., 2019) Arabic language model, and images with ViT-base (Dosovitskiy et al., 2021); each encoder's pooled output is concatenated and passed through a classification head (Linear → GELU → Dropout → Linear) to produce the final logits.

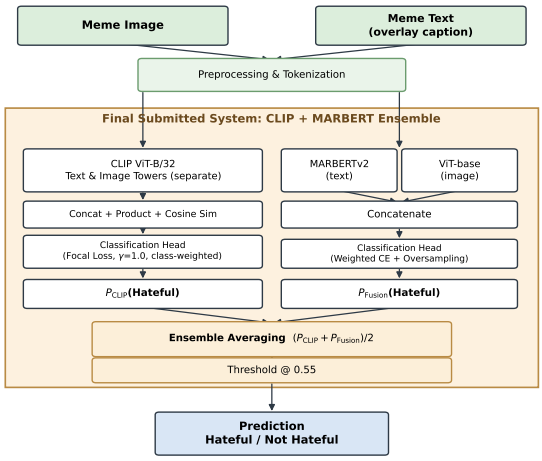


Figure 1: Workflow of the final CLIP-MARBERT ensemble system.

The model is trained with AdamW (Loshchilov and Hutter, 2019) (learning rate 2×10^{-5}, weight decay 0.01) under a linear schedule with 6% warmup, using class-weighted cross-entropy loss. Training batches are drawn with a weighted random sampler that oversamples the minority Hateful class, for up to 8 epochs with batch size 16 and early stopping (patience of 2 epochs without improvement) based on Macro-F1 on `dev`. Predictions on `dev_test` use the default 0.5 cutoff on the positive (Hateful) class probability; no threshold sweep was performed for this system.

3.2 Final Submitted System: CLIP-MARBERT Ensemble

The final test-set submission combines two independently trained classifiers whose predicted probabilities are averaged at inference time.

Branch 1 (CLIP-based classifier). Text and image are encoded separately through CLIP ViT-B/32's (Radford et al., 2021) native text and image towers, producing L2-normalized text and image embeddings. These are combined into a single fusion vector consisting of the text embedding, the image embedding, their element-wise product, and their cosine similarity. The resulting vector is passed through a two-layer MLP classification head (GELU activation, dropout), trained with inverse-frequency-weighted focal loss (Lin et al., 2017) ($\gamma = 1.0$).

Branch 2 (late-fusion classifier). The same architecture as the development-phase system: MARBERTv2 and ViT-base encoders, concatenated and

passed to a classification head, trained with class-weighted cross-entropy loss and a weighted random sampler that oversamples the minority Hateful class.

Both branches are retrained with AdamW (learning rate 2×10^{-5}, weight decay 0.01) under a linear schedule with 6% warmup, on the union of the `train`, `dev`, and `dev_test` partitions, so the final model benefits from all released gold labels before predicting on the blind test set. The CLIP branch is trained for 8 epochs and the late-fusion branch for 2, reflecting their differing convergence behavior in preliminary runs. At inference time, the two branches' softmax probabilities are averaged, and instances with an averaged Hateful probability at or above 0.55 are labelled Hateful; as with the development-phase system, this threshold was set by manual inspection of a small number of development predictions rather than a systematic sweep. Table 2 summarizes the key hyperparameters for both systems.

Hyperparameter	Dev system	Final system
Learning rate	2e-5	2e-5
Weight decay	0.01	0.01
Batch size	16	16
Epochs	$\leq$8 (early stop)	8 / 2
Loss	Weighted CE	Focal (γ=1) / WCE
Schedule	Linear	Linear
Max text length	128	128 / 77
Threshold	Fixed (0.5)	Fixed (0.55)

Table 2: Hyperparameter configuration for both systems.

The pretrained checkpoints used are `UBC-NLP/MARBERTv2` for MARBERT, `google/vit-base-patch16-224` for ViT, and `openai/clip-vit-base-patch32` for CLIP, all obtained from the Hugging Face Hub. All experiments were run on a Colab GPU instance using PyTorch and the Hugging Face `transformers` library, with random seed 42 fixed for both systems.

4 Experimental Results

The final CLIP-MARBERT ensemble substantially outperforms the development-phase system across all metrics. When trained on `train+dev+dev_test` and evaluated on the official blind test set, it obtains a Macro-F1 of 0.7457 and an accuracy of 0.7680, compared to 0.6554 and 0.6860, respectively, for the development-phase system trained only on `train` and evaluated on `dev_test`, an absolute improvement of 9.03 Macro-F1 points. On the official Subtask A1 leaderboard, this places team Bahash-AI (submitted under the leaderboard username `ahmadullah04`) 12th of 16 participating teams, above the AraBERTv2 baseline (0.678 Macro-F1) released by the organizers.

This comparison should not be read as a controlled ablation. The two systems differ simultaneously in architecture, training-set size, decision threshold, and evaluation split. On data alone, the final system trains on `train+dev+dev_test` (4,500 labelled instances) against the development system's `train` alone (3,500 instances), approximately 1.29 times as much labelled data, not a larger multiple. This gain likely reflects a combination of factors: the added CLIP branch, the larger training set available to the final model, and the more stable decision boundary that typically results from ensembling two independently trained classifiers. Full results for both systems are presented in Table 3.

Metric	Dev system (`dev_test`)	Final system (blind test)
Macro-F1	0.6554	**0.7457**
Accuracy	0.6860	0.7680
Macro precision	0.6633	0.7385
Macro recall	0.6521	0.7765
Weighted F1	0.6805	0.7764

Table 3: Official results for both systems.

Table 4 breaks results down by class using precision, recall, and F1 from the official scorer. For both systems, F1 on the Not Hateful class exceeds F1 on the Hateful class by a wide margin, a pattern consistent with the underlying class imbalance. This gap narrows only modestly between the two systems despite the final system's stronger overall Macro-F1. The precision and recall columns, discussed in Section 5, show that this similar F1 gap arises from opposite underlying behavior in the two systems.

Class / System	P	R	F1
Hateful (dev system)	0.599	0.513	0.553
Not Hateful (dev system)	0.728	0.791	0.758
Hateful (final system)	0.578	0.797	0.670
Not Hateful (final system)	0.899	0.756	0.821

Table 4: Per-class precision, recall, and F1 for both systems.

The final system's Macro-F1 of 0.7457 represents a substantial improvement over the development-phase baseline, achieved without any additional external annotation beyond the officially released ArGuard splits.

5 Error Analysis

Two patterns are consistent across both systems and the released gold labels.

Blind-test performance exceeds development-phase performance. This is not the result of leakage: the final system's evaluation split (blind test) is entirely disjoint from its `train+dev+dev_test` training data. Rather, it stems from the design differences discussed in Section 4: additional training data, an added CLIP branch, and ensembling. These factors are confounded in our current experiments, and we recommend that future iterations isolate them through controlled ablations.

Recall and precision trade off in opposite directions. Table 4 shows the two systems reach a comparable Hateful-class F1 gap relative to Not Hateful, but through opposite error patterns. The development-phase system is more conservative: it misses more hateful memes than it incorrectly flags, with recall (0.513) trailing precision (0.599). The final ensemble reverses this pattern: recall rises to 0.797 while precision falls to 0.578, so it catches more hateful content but at the cost of mislabeling a larger share of Not Hateful memes. This shift is consistent with the final system's decision threshold (0.55, only slightly above chance) combined with the added CLIP branch and larger training set, though we cannot isolate which factor dominates. Which behavior is preferable depends on the deployment context: higher recall is preferable when missing hateful content is costly, while higher precision is preferable when over-flagging legitimate speech is costly. We did not conduct a manual review of individual false positives and false negatives from the released gold labels for this submission; doing so would clarify whether the final system's false positives cluster around specific content types (see Limitations).

6 Conclusion

We presented two related multimodal systems for ArGuard Subtask A1, binary hateful meme detection in Arabic. Our development-phase system, a late-fusion classifier combining MARBERTv2 and ViT-base, reached a Macro-F1 of 0.6554 on the development-test split. This same classifier was retained as one branch of our final submitted system, an ensemble with an added CLIP-based branch trained on all available labelled data, which reached 0.7457 on the blind test set, ranking 12th of 16 teams. Across both systems, class-imbalance handling through class-weighted losses, oversampling, and threshold selection contributed alongside the added CLIP branch and larger final training set, though our experiments do not isolate the individual effect of each. Per-class precision and recall reveal that the two systems trade off differently on the Hateful class: the development-phase system under-recalls, while the final ensemble over-predicts Hateful at the cost of precision. Future work includes a controlled ablation study, a manual error review using the now-public gold labels, explicit modelling of OCR text embedded in images, and a more systematically validated decision threshold.

Limitations

We used a single random seed (42) per system due to compute constraints, so our scores do not reflect run-to-run variance, and we did not test significance against leaderboard baselines. Neither system's decision threshold was chosen via a sweep validated on a held-out split: the development-phase system uses the untuned default cutoff of 0.5, and the final ensemble's threshold (0.55) was set by manual inspection of a small number of development predictions. The development-phase system is evaluated on `dev_test` rather than `dev`, while the final ensemble's threshold was informed by predictions on `dev`; both splits are of equal size but this distinction should be kept in mind when comparing the two systems. Because the final model trains on `train+dev+dev_test` combined, we lack a clean, leak-free score for it; the development-phase number reported here comes from an earlier, standalone run of the same Branch 2 architecture, evaluated before the CLIP branch was added and before additional training data was incorporated. The reported improvement should therefore not be interpreted as a controlled ablation, as multiple factors change simultaneously. Our error analysis identifies a precision-recall trade-off using aggregate per-class statistics but does not include a manual review of individual misclassified examples, despite gold test labels now being publicly available. All training ran on a single consumer-grade Colab

GPU, which limited batch size, model scale, and the encoder combinations we could explore.

Ethical Considerations

This work models hateful content in Arabic memes for content-moderation research, using the officially released AHA-Memes benchmark (Kmainasi et al., 2026). We did not collect, annotate, or redistribute additional data. The two systems in this paper fail in different ways: the development-phase system's lower recall means it will miss a nontrivial share of actual hateful content, while the final system's lower precision means it will flag a nontrivial share of legitimate speech, satire, or counter-speech as hateful. Both failure modes carry real costs, and we do not recommend either system for unsupervised, automated removal without human review. We made no attempt to identify or de-anonymize individuals in the images, and no outputs from this work are intended for profiling individuals or communities. Hateful-labelled examples were viewed only in aggregate during development and are not reproduced in this text.

Acknowledgments

We thank the ArGuard organizers for releasing AHA-Memes, maintaining the CodaBench leaderboard, and providing gold test labels for post-task analysis.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Firoj Alam, Md. Rafiul Biswas, Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Hamdy Mubarak, Georgios Mikros, Abul Hasnat, and Wajdi Zaghouani. 2026. ArGuard Shared Task: Harmful content detection in Arabic Memes and LLM Prompts. In *Proceedings of the Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*.

Haibo He and Edwardo A. Garcia. 2009. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284.

Douwe Kiela, Hamed Firooz, Aravind Mohan, Vedanuj Goswami, Amanpreet Singh, Pratik Ringshia, and Davide Testuggine. 2020. The hateful memes challenge: Detecting hate speech in multimodal memes. In *Advances in Neural Information Processing Systems*, volume 33, pages 2611–2624.

Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Abul Hasnat, Md. Rafiul Biswas, Wajdi Zaghouani, and Firoj Alam. 2026. AHA-Memes: A fine-grained multimodal benchmark for understanding hate in arabic memes. *arXiv preprint arXiv:2607.27393*.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*.